

Python Final Exam Preparation — 150 Questions

Topics Covered: Python Starter Kits · All Datatypes · Control Flow · Loops · Functions & Parameters · OOP · Lambda Functions · Wrappers & Decorators · Regular Expressions · MRO · Context Manager Protocol

Section 1: Python Starter Kits (Q1-Q12)

Q1. What is the output of the following code?

```
python

print(type(10), type(10.0), type("10"))
```

Answer: `<class 'int'>` `<class 'float'>` `<class 'str'>`

Q2. What is the difference between `==` and `is` in Python?

Answer: `==` checks value equality; `is` checks identity (whether both variables point to the same object in memory).

Q3. What will the following print?

```
python

x = 5
y = x
x = 10
print(y)
```

Answer: `5` — integers are immutable; `y` still holds the original value.

Q4. What is the purpose of `if __name__ == "__main__":` in a Python script?

Answer: It ensures the code block runs only when the file is executed directly, not when it is imported as a module.

Q5. What are Python's built-in numeric types?

Answer: `int`, `float`, `complex`

Q6. What is the output?

```
python  
  
print(10 // 3, 10 % 3, 2 ** 10)
```

Answer: `3 1 1024`

Q7. How do you check the Python version from the command line?

Answer: `python --version` or `python3 --version`

Q8. What does the `pass` statement do?

Answer: It is a null operation; used as a placeholder where a statement is syntactically required but no action is needed.

Q9. What is the difference between a script and a module in Python?

Answer: A script is run directly; a module is imported by other files. The same file can be both.

Q10. What is the output of `bool(0), bool(""), bool([]), bool(None)`?

Answer: `False False False False` — all are falsy values.

Q11. What does `help()` do in Python?

Answer: Displays documentation for a Python object, module, function, or keyword.

Q12. What is PEP 8?

Answer: PEP 8 is Python's official style guide covering naming conventions, indentation (4 spaces), line length, imports, and other coding standards.

■ Section 2: All Datatypes (Q13-Q28)

Q13. List all built-in data types in Python.

Answer: `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `set`, `frozenset`, `dict`, `bytes`, `bytearray`, `memoryview`, `NoneType`

Q14. What is the difference between a `list` and a `tuple`?

Answer: Lists are mutable (can be changed); tuples are immutable (cannot be changed after creation). Tuples are generally faster and can be used as dictionary keys.

Q15. What is the output?

```
python
s = {1, 2, 2, 3, 3, 3}
print(s)
```

Answer: `{1, 2, 3}` — sets contain only unique elements.

Q16. What is a `frozenset` and when would you use it?

Answer: A `frozenset` is an immutable version of a set. Use it when you need a set that cannot change, e.g., as a dictionary key or set element.

Q17. What is the difference between `append()` and `extend()` on a list?

Answer: `append()` adds a single element; `extend()` adds all elements of an iterable.

Q18. What is the output?

```
python
d = {"a": 1, "b": 2}
print(d.get("c", 99))
```

Answer: `99` — `get()` returns the default value when the key is missing.

Q19. How do you create an empty dictionary vs. an empty set?

Answer: Empty dict: `{}` or `dict()`. Empty set: `set()` — you **cannot** use `{}` for an empty set.

Q20. What is the output of `"hello"[1:4]`?

Answer: `'ell'`

Q21. What is string interning in Python?

Answer: Python caches small strings and integers. Two variables assigned the same string literal may point to the same object, making `is` return `True`.

Q22. What is the difference between `bytes` and `bytearray`?

Answer: `bytes` is immutable; `bytearray` is mutable. Both store sequences of integers in the range 0-255.

Q23. What does `list(range(1, 10, 2))` produce?

Answer: `[1, 3, 5, 7, 9]`

Q24. What is the output?

```
python
t = (1,)
print(type(t))
```

Answer: `<class 'tuple'>` — a trailing comma makes it a tuple.

Q25. How do you merge two dictionaries in Python 3.9+?

Answer: Use the `|` operator: `merged = dict1 | dict2`

Q26. What is a `memoryview` object?

Answer: A `memoryview` exposes the internal buffer of an object (like `bytes` or `bytearray`) without copying, enabling efficient slicing and manipulation of large data.

Q27. What is the difference between `del`, `.remove()`, and `.pop()` for lists?

Answer: `del` removes by index; `.remove()` removes by value (first occurrence); `.pop()` removes by index and returns the removed element (defaults to last).

Q28. What is the output?

```
python
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Answer: `[1, 2, 3, 4]` — `b` is not a copy; both variables reference the same list.

Section 3: Control Flow (Q29–Q43)

Q29. What are Python's conditional statements?

Answer: `if`, `elif`, `else`

Q30. What is a ternary expression in Python? Give an example.

Answer: `value = "even" if x % 2 == 0 else "odd"` — an inline if-else expression.

Q31. What is the output?

```
python
```

```
x = 15
if x > 10:
    if x > 20:
        print("A")
    else:
        print("B")
else:
    print("C")
```

Answer: **B**

Q32. What is the difference between **break** and **continue**?

Answer: **break** exits the loop entirely; **continue** skips the current iteration and moves to the next.

Q33. What does the **else** clause on a loop do?

Answer: It runs when the loop completes without hitting a **break** statement.

Q34. What is the output?

```
python

for i in range(5):
    if i == 3:
        break
else:
    print("Done")
print("End")
```

Answer: **End** — the **else** block is skipped because **break** was hit.

Q35. What is a **match** statement (Python 3.10+)?

Answer: Structural pattern matching similar to switch-case. It matches a value against patterns and executes the first matching branch.

Q36. Write a match statement to check the day type.

Answer: Prints `Weekend`

Q37. What is short-circuit evaluation?

Answer: In `and` / `or` expressions, Python stops evaluating as soon as the result is determined. `False and X` skips `X`; `True or X` skips `X`.

Q38. What is the output?

```
python
print(0 or "hello" or "world")
```

Answer: `'hello'` — `or` returns the first truthy value.

Q39. What is the output?

```
python
print(1 and 2 and 3)
```

Answer: `3` — `and` returns the last value when all are truthy.

Q40. How does `assert` work?

Answer: `assert condition, "message"` raises an `AssertionError` if the condition is `False`.
Used for debugging and sanity checks.

Q41. What is the difference between `try/except` and `try/finally`?

Answer: `except` handles specific exceptions; `finally` always runs regardless of whether an exception occurred — used for cleanup.

Q42. What is the output?

```
python

try:
    1 / 0
except ZeroDivisionError:
    print("Caught")
finally:
    print("Always")
```

Answer: `Caught` then `Always`

Q43. Can you have multiple `except` clauses? Give an example.

Answer: Yes.

```
python

try:
    int("abc")
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
```

■ Section 4: Loops (Q44-Q56)

Q44. What is the difference between `for` and `while` loops?

Answer: `for` iterates over a sequence/iterable; `while` runs as long as a condition is `True`.

Q45. What does `enumerate()` do?

Answer: Returns an iterator of `(index, value)` pairs from an iterable.

```
python
```



```
for i, v in enumerate(["a", "b", "c"]):  
    print(i, v)
```

Q46. What is the output?

```
python  
  
for i in range(3):  
    for j in range(3):  
        if i == j:  
            print(i, j)
```

Answer: (0 0), (1 1), (2 2)

Q47. What is a list comprehension? Write one that squares even numbers from 1-10.

Answer: [x**2 for x in range(1, 11) if x % 2 == 0] → [4, 16, 36, 64, 100]

Q48. What is a dictionary comprehension? Give an example.

Answer: {k: v for k, v in zip("abc", [1, 2, 3])} → {'a': 1, 'b': 2, 'c': 3}

Q49. What is a generator expression? How does it differ from a list comprehension?

Answer: Uses `()` instead of `[]`. It yields values lazily (one at a time), consuming less memory. Example: `(x**2 for x in range(100))`

Q50. What does `zip()` do?

Answer: Combines multiple iterables element-wise, returning tuples. Stops at the shortest iterable.

Q51. What is the output?

```
python  
  
for x, y in zip([1, 2, 3], [4, 5]):  
    print(x + y)
```

Answer: 5 then 7 — stops at the shortest iterable.

Q52. How do you loop over a dictionary's key-value pairs?

Answer: `for k, v in my_dict.items():`

Q53. What is `itertools.chain()`?

Answer: Chains multiple iterables together so they can be iterated as one sequence.

Q54. What does `while True: ... break` pattern accomplish?

Answer: Creates an infinite loop that exits only when `break` is hit — often used for menus or polling.

Q55. What is the output?

```
python
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue
    print(i, end=" ")
```

Answer: 1 2 4 5

Q56. What is a nested list comprehension? Flatten `[[1,2],[3,4]]`.

Answer: `[x for row in [[1,2],[3,4]] for x in row]` → `[1, 2, 3, 4]`

Section 5: Functions & Function Parameters (Q57-Q73)

Q57. What are the four types of function arguments in Python?

Answer: Positional, keyword, `*args` (variable positional), `**kwargs` (variable keyword).

Q58. What is the difference between `*args` and `**kwargs`?

Answer: `*args` collects extra positional arguments as a tuple; `**kwargs` collects extra keyword arguments as a dictionary.

Q59. What is the output?

```
python

def f(a, b=10, *args, **kwargs):
    print(a, b, args, kwargs)

f(1, 2, 3, 4, x=5)
```

Answer: `1 2 (3, 4) {'x': 5}`

Q60. What is a keyword-only argument?

Answer: An argument that must be passed by name, defined after `*` or `*args`.

```
python

def f(a, *, b):
    print(a, b)

f(1, b=2) # valid
```

Q61. What is a positional-only argument (Python 3.8+)?

Answer: Defined before `/` in the signature. Can only be passed positionally.

```
python

def f(x, /, y):
    ...
```

Q62. What is the mutable default argument trap?

```
python
```

```
def append_to(val, lst=[]):  
    lst.append(val)  
    return lst
```

Answer: The default `lst=[]` is created once and shared across all calls. Use `lst=None` and set `lst = []` inside the function to fix.

Q63. What is a first-class function in Python?

Answer: Functions are objects — they can be stored in variables, passed as arguments, and returned from other functions.

Q64. What is a higher-order function? Give an example.

Answer: A function that takes or returns another function. Examples: `map()`, `filter()`, `sorted()` with a `key`.

Q65. What does `*` in a function call do?

Answer: Unpacks a sequence into positional arguments: `f(*[1, 2, 3])` is `f(1, 2, 3)`.

Q66. What does `**` in a function call do?

Answer: Unpacks a dictionary into keyword arguments: `f(**{"a": 1})` is `f(a=1)`.

Q67. What is a closure in Python?

Answer: A function that captures and remembers variables from its enclosing scope even after the outer function has returned.

Q68. Write a closure that returns a multiplier function.

```
python
```

```
def make_multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = make_multiplier(2)
print(double(5))  # 10
```

Q69. What is the `global` keyword used for?

Answer: Declares that a variable inside a function refers to the global scope rather than creating a local variable.

Q70. What is the `nonlocal` keyword?

Answer: Refers to the nearest enclosing scope (not global) variable, allowing a nested function to modify it.

Q71. What is a recursive function? What is the risk?

Answer: A function that calls itself. The risk is hitting Python's recursion limit (`sys.getrecursionlimit()`, default 1000) causing a `RecursionError`.

Q72. What does `functools.lru_cache` do?

Answer: Caches function results based on arguments (memoization), improving performance for repeated calls with the same inputs.

Q73. What is the difference between `return` and `yield`?

Answer: `return` exits the function and returns a value; `yield` pauses execution and returns a value lazily, making the function a generator.

■ Section 6: Lambda Functions (Q74-Q83)

Q74. What is a lambda function?

Answer: An anonymous, single-expression function: `lambda arguments: expression`

Q75. What is the output?

```
python

f = lambda x, y: x ** y
print(f(2, 8))
```

Answer: `256`

Q76. Can a lambda have multiple statements?

Answer: No. Lambdas are limited to a single expression. Use a regular `def` for multi-statement logic.

Q77. How do you sort a list of tuples by the second element using a lambda?

Answer: `sorted(data, key=lambda x: x[1])`

Q78. Use `map()` with a lambda to square a list.

Answer: `list(map(lambda x: x**2, [1, 2, 3, 4]))` → `[1, 4, 9, 16]`

Q79. Use `filter()` with a lambda to keep even numbers.

Answer: `list(filter(lambda x: x % 2 == 0, range(10)))` → `[0, 2, 4, 6, 8]`

Q80. Use `reduce()` to compute the product of a list.

```
python

from functools import reduce
reduce(lambda x, y: x * y, [1, 2, 3, 4, 5]) # → 120
```

Q81. When should you prefer a lambda over a regular function?

Answer: For short, single-use expressions passed to higher-order functions. For anything complex or reused, a named `def` is cleaner.

Q82. What is the output?

```
python

funcs = [lambda x: x + i for i in range(3)]
print([f(0) for f in funcs])
```

Answer: `[2, 2, 2]` — all lambdas share the same `i`, which is `2` after the loop ends (late binding).

Q83. How do you fix the late-binding issue in lambdas?

Answer: Use a default argument to capture the value: `lambda x, i=i: x + i`

Section 7: Object-Oriented Programming (Q84-107)

Q84. What are the four pillars of OOP?

Answer: Encapsulation, Inheritance, Polymorphism, Abstraction.

Q85. What is the difference between a class and an instance?

Answer: A class is a blueprint; an instance is a concrete object created from that blueprint.

Q86. What is `__init__`?

Answer: The initializer (constructor) called when a new instance is created. It sets up instance attributes.

Q87. What is `self`?

Answer: A reference to the current instance of the class. It must be the first parameter of instance methods.

Q88. What is the difference between class attributes and instance attributes?

Answer: Class attributes are shared across all instances; instance attributes are unique to each object.

Q89. What are `@classmethod` and `@staticmethod`?

Answer: `@classmethod` receives the class (`cls`) as the first argument; `@staticmethod` receives no implicit first argument. Neither operates on `self`.

Q90. What is encapsulation? How is it achieved in Python?

Answer: Hiding internal state. Python uses naming conventions: `_name` (protected, soft) and `__name` (private, name-mangled to `__ClassName__name`).

Q91. What is inheritance? Show a basic example.

```
python

class Animal:
    def speak(self): return "..."

class Dog(Animal):
    def speak(self): return "Woof"
```

Q92. What is `super()`?

Answer: Calls the parent class's method. Used in `__init__` and overridden methods to avoid hard-coding the parent name.

Q93. What is multiple inheritance?

Answer: A class inheriting from more than one parent: `class C(A, B): ...`

Q94. What are dunder (magic) methods? List five.

Answer: Special methods with double underscores: `__init__`, `__str__`, `__repr__`, `__len__`, `__eq__`, `__add__`, `__iter__`, `__enter__`, `__exit__`.

Q95. What is the difference between `__str__` and `__repr__`?

Answer: `__str__` is human-readable; `__repr__` is for developers/debugging and should ideally be unambiguous. `repr()` calls `__repr__`; `str()` calls `__str__`.

Q96. What is operator overloading? Give an example.

Answer: Defining dunder methods to support operators on custom objects. E.g., defining `__add__` to support `+`.

Q97. What is `@property`?

Answer: A decorator that defines a getter method accessible like an attribute. Pair with `@x.setter` and `@x.deleter` for full property control.

Q98. What is an abstract class?

Answer: A class with one or more abstract methods that subclasses must implement, defined using `abc.ABC` and `@abstractmethod`.

Q99. What is the difference between composition and inheritance?

Answer: Inheritance is "is-a"; composition is "has-a". Composition is often preferred for flexibility.

Q100. What is polymorphism in Python?

Answer: The same interface working with different types. E.g., `len()` works on strings, lists, dicts — each implementing `__len__`.

Q101. What does `isinstance()` do?

Answer: Checks if an object is an instance of a class or its subclasses: `isinstance(obj, MyClass)`

Q102. What is `__slots__`?

Answer: A class-level declaration that restricts instance attributes, reducing memory usage by replacing the default `__dict__`.

Q103. What is a dataclass (Python 3.7+)?

Answer: `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` based on annotated fields, reducing boilerplate.

Q104. What is the difference between `__new__` and `__init__`?

Answer: `__new__` creates the instance (allocates memory); `__init__` initializes it. Override `__new__` when subclassing immutables like `int` or `str`.

Q105. What is method resolution order (MRO)?

Answer: The order in which Python searches for a method in a class hierarchy. Python uses the C3 linearization algorithm.

Q106. What is a mixin?

Answer: A class designed to add specific functionality to other classes via multiple inheritance, without being a standalone class.

Q107. What does `__call__` do?

Answer: Makes an instance callable like a function: `obj()` triggers `__call__`.

12 34 Section 8: MRO — Method Resolution Order (Q108-Q116)

Q108. What algorithm does Python use for MRO?

Answer: C3 Linearization (C3 superclass linearization), introduced in Python 2.3.

Q109. How do you inspect a class's MRO?

Answer: `ClassName.__mro__` or `ClassName.mro()` or `help(ClassName)`.

Q110. What is the "diamond problem" in OOP?

Q111. What is the output?

```
python

class A:
    def greet(self): return "A"
class B(A):
    def greet(self): return "B"
class C(A):
    def greet(self): return "C"
class D(B, C):
    pass

print(D().greet())
print(D.__mro__)
```

Answer: `B`; MRO: `D → B → C → A → object`

Q112. In what order does `super()` call methods in a cooperative multiple inheritance chain?

Answer: It follows the MRO. Each `super()` call delegates to the next class in the MRO, not necessarily the immediate parent.

Q113. What is `object` in Python's class hierarchy?

Answer: Every class in Python 3 implicitly inherits from `object`. It's the root of all class hierarchies.

Q114. What happens if two parent classes define the same method and neither calls `super()`?

Answer: Only the method from the first class in MRO is used. The second class's method is skipped.

Q115. When does Python raise a `TypeError` related to MRO?

Answer: When it cannot construct a consistent MRO — e.g., conflicting inheritance orders that violate C3 rules.

Q116. What is cooperative multiple inheritance?

Answer: All classes in the hierarchy call `super()` so every class in the MRO gets an opportunity to run its method.

🔑 Section 9: Wrappers & Decorators (Q117-Q129)

Q117. What is a decorator?

Answer: A function (or class) that wraps another function to extend or modify its behavior without changing its source code. Applied with `@decorator_name`.

Q118. Write a simple decorator that prints "Before" and "After".

```
python

def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper

@my_decorator
def say_hello():
    print("Hello")
```

Q119. What problem does `functools.wraps` solve?

Answer: Without it, the wrapped function loses its `__name__`, `__doc__`, and other metadata. `@functools.wraps(func)` preserves them.

Q120. What is a decorator factory (parameterized decorator)?

Answer: A function that takes arguments and returns a decorator.

```
python
```

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def hi(): print("Hi")
```

Q121. Can you stack multiple decorators? What is the order of application?

Answer: Yes. They are applied bottom-up (closest to the function first), but executed top-down.

```
python

@A
@B
def f(): ...

# Equivalent to: f = A(B(f))
```

Q122. What is a class-based decorator?

Answer: A class that implements `__call__` to act as a decorator.

```
python

class Timer:
    def __init__(self, func):
        self.func = func
    def __call__(self, *args, **kwargs):
        import time
        start = time.time()
        result = self.func(*args, **kwargs)
        print(f"Elapsed: {time.time() - start:.4f}s")
        return result
```

Q123. What is `@property` as a decorator?

Answer: Turns a method into a read-only attribute. Combined with `@x.setter` allows controlled attribute access.

Q124. What built-in decorators does Python provide?

Answer: `@staticmethod`, `@classmethod`, `@property`, `@functools.lru_cache`, `@functools.cache`, `@functools.wraps`, `@dataclasses.dataclass`, `@abstractmethod`

Q125. Write a memoization decorator from scratch.

```
python

def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper
```

Q126. What is the difference between `@functools.lru_cache` and `@functools.cache`?

Answer: `lru_cache` has a bounded size (default 128); `cache` (Python 3.9+) is unbounded and slightly faster.

Q127. How do decorators relate to the Decorator design pattern?

Answer: Python decorators implement the Decorator pattern — wrapping objects/functions to add behavior dynamically without modifying the original.

Q128. What is a context decorator?

Answer: A decorator created with `contextlib.contextmanager` that can also act as a context manager using `with`.

Q129. How would you write a decorator that logs arguments and return values?

Section 10: Context Manager Protocol (Q130–Q139)

Q130. What is the Context Manager Protocol?

Answer: An object implementing `__enter__` and `__exit__` methods, enabling use with the `with` statement for resource management.

Q131. What does the `with` statement guarantee?

Answer: `__exit__` is always called when the block ends — whether normally or due to an exception — ensuring proper cleanup (file close, lock release, etc.).

Q132. What is the signature of `__exit__`?

Answer: `def __exit__(self, exc_type, exc_val, exc_tb):` — receives exception info. Return `True` to suppress the exception; `None`/`False` to propagate it.

Q133. Write a class-based context manager for timing code.

```
python

import time

class Timer:
    def __enter__(self):
        self.start = time.time()
        return self
    def __exit__(self, *args):
        self.elapsed = time.time() - self.start
        print(f"Elapsed: {self.elapsed:.4f}s")
```

Q134. What is `contextlib.contextmanager`?

Answer: A decorator that turns a generator function into a context manager. Code before `yield` is `__enter__`; code after is `__exit__`.

```
python

from contextlib import contextmanager

@contextmanager
def open_resource():
    print("Acquiring")
    yield "resource"
    print("Releasing")
```

Q135. What is `contextlib.suppress`?

Answer: A context manager that silently suppresses specified exceptions.

```
python

with contextlib.suppress(FileNotFoundError):
    os.remove("missing.txt")
```

Q136. What is `contextlib.ExitStack`?

Answer: Dynamically manages multiple context managers — useful when the number of contexts is unknown at write time.

Q137. What happens if `__exit__` returns `True`?

Answer: The exception raised inside the `with` block is suppressed (swallowed).

Q138. Can you nest `with` statements?

Answer: Yes. Python 3 supports multiple context managers in one line: `with A() as a, B()`
`as b:`

Q139. When would you write a custom context manager instead of using `try/finally`?

Answer: When the cleanup logic is reusable across multiple places. Context managers are cleaner, more Pythonic, and can be used with `with` for readability.

Section 11: Regular Expressions (Q140–150)

Q140. What module provides regular expressions in Python?

Answer: `re`

Q141. What is the difference between `re.match()` and `re.search()`?

Answer: `match()` checks only at the beginning of the string; `search()` scans the entire string for a match.

Q142. What is the output?

```
python

import re
print(bool(re.match(r"\d+", "123abc")))
print(bool(re.match(r"\d+", "abc123")))
```

Answer: `True` then `False`

Q143. What do the following metacharacters mean: `.` `^` `$` `*` `+` `?` `{}`?

Answer:

- `.` — any character except newline
 - `^` — start of string
 - `$` — end of string
 - `*` — 0 or more
 - `+` — 1 or more
 - `?` — 0 or 1
 - `{n,m}` — between n and m repetitions
-

Q144. What is a character class? Give examples.

Answer: `[abc]` matches a, b, or c. `[a-z]` matches lowercase letters. `[^0-9]` matches any non-digit.

Q145. What are `\d`, `\w`, `\s` and their uppercase counterparts?

Answer:

- `\d` — digit; `\D` — non-digit
 - `\w` — word character (letter/digit/underscore); `\W` — non-word
 - `\s` — whitespace; `\S` — non-whitespace
-

Q146. What is a capturing group? What is a non-capturing group?

Answer: `(pattern)` — capturing group, stores the match. `(?:pattern)` — non-capturing group, groups without storing.

Q147. How do you extract all matches from a string?

Answer: `re.findall(pattern, string)` returns a list of all matches.

```
python
re.findall(r"\d+", "abc 12 def 34") # ['12', '34']
```

Q148. What is `re.sub()`? Give an example.

Answer: Replaces all occurrences of a pattern.

```
python
re.sub(r"\s+", "-", "hello world foo") # 'hello-world-foo'
```

Q149. What is the `re.IGNORECASE` flag?

Answer: Makes the pattern case-insensitive. Alias: `re.I`.

```
python
re.search(r"hello", "Hello World", re.I)
```

Q150. What is a lookahead and lookbehind in regex?

Answer:

- **Positive lookahead:** `\d+(?= dollars)` — matches digits followed by " dollars" (without consuming " dollars")
 - **Negative lookahead:** `\d+(?! dollars)` — matches digits NOT followed by " dollars"
 - **Positive lookbehind:** `(?<=\d)\d+` — matches digits preceded by `\d`
 - **Negative lookbehind:** `(?<!\d)\d+` — matches digits NOT preceded by `\d`
-

Quick Reference Summary

Section	Questions	Topics
Python Starter Kits	Q1-Q12	Basics, operators, PEP 8
All Datatypes	Q13-Q28	int, str, list, dict, set, tuple...
Control Flow	Q29-Q43	if/elif/else, match, try/except
Loops	Q44-Q56	for, while, comprehensions, generators
Functions & Parameters	Q57-Q73	args, kwargs, closures, recursion
Lambda Functions	Q74-Q83	map, filter, reduce, late binding
OOP	Q84-Q107	Classes, inheritance, magic methods
MRO	Q108-Q116	C3, diamond problem, super()
Wrappers & Decorators	Q117-Q129	functools, stacking, class decorators
Context Managers	Q130-Q139	with, enter , exit , contextlib
Regular Expressions	Q140-Q150	re module, patterns, groups, flags

Exam Tips:

- Always trace code execution step by step before selecting an answer.
- Know the difference between mutable and immutable types cold.
- Practice writing decorators and context managers from scratch.
- Understand MRO by drawing class diagrams for diamond problems.

- For regex, build patterns incrementally and test with `re.findall`.

Good luck on your final exam! 🎓